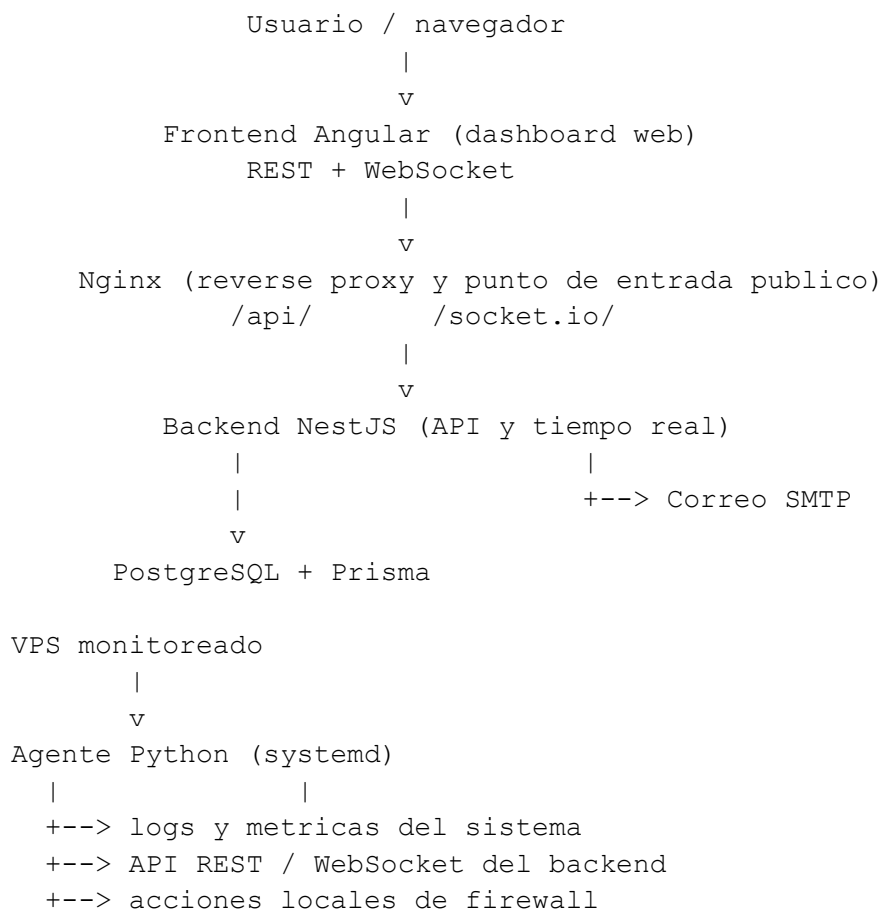


LUMI Guardian AI: arquitectura y tecnologías

Este documento resume la arquitectura del monorepo, la responsabilidad de sus directorios y las tecnologías declaradas en cada proyecto.

1. Arquitectura general

LUMI se compone de tres aplicaciones principales y una capa de infraestructura:



Componentes y responsabilidades

Componente	Responsabilidad	Ejecucion
<code>client/</code>	Dashboard, autenticacion, gestion de VPS, alertas, bloqueos y configuracion	Navegador; build estatico de Angular
<code>server/</code>	API REST, autenticacion, reglas de autonomia, persistencia, correo y WebSocket	Node.js; localmente o en Docker
<code>agent/</code>	Monitoreo del VPS, envio de telemetria/alertas y ejecucion de bloqueos locales	Python en cada VPS, normalmente como <code>systemd</code>
PostgreSQL	Persistencia de usuarios, VPS, alertas, bloqueos, metricas, configuracion y auditoria	Contenedor Docker o servicio administrado
Nginx	Sirve el frontend y enruta API, instalador del agente y Socket.IO	Contenedor en despliegue

Flujos principales

1. El usuario inicia sesion en Angular. El backend gestiona `accessToken` JWT y `refreshToken` mediante `cookie HttpOnly`.
2. El agente se identifica ante el backend con un token propio asociado a un VPS.
3. El agente analiza eventos del sistema y envia alertas y metricas al backend por HTTP/HTTPS o Socket.IO.
4. El backend persiste los datos con Prisma en PostgreSQL y aplica el nivel de autonomia configurado.
5. Cuando corresponde, el backend solicita al agente bloquear o desbloquear una IP.
6. El backend publica nuevas alertas en el namespace `/dashboard` y el frontend actualiza el dashboard en tiempo real.
7. Nginx concentra el origen publico: `/` hacia Angular, `/api/` hacia NestJS y `/socket.io/` hacia Socket.IO.

2. Arquitectura por directorios

Directorios de la raiz

Directorio/archivo	Proposito
agent/	Proyecto Python instalado en los VPS monitoreados. Incluye monitores, configuracion, envio de datos y control local.
client/	Aplicacion web Angular del dashboard.
server/	API NestJS, logica de negocio, WebSocket y acceso a PostgreSQL.
deploy/	Compose y configuracion para desplegar frontend, backend, PostgreSQL y Nginx como sistema central.
docs/	Documentacion funcional, tecnica, contratos API, base de datos, pruebas y arquitectura.
test-ws/	Utilidades JavaScript para probar los canales WebSocket con un listener de dashboard y un agente simulado.
nginx/	Directorio reservado para configuracion de Nginx a nivel de raiz; actualmente no contiene archivos. El Nginx productivo esta en <code>deploy/nginx/</code> .
<code>docker-compose.yml</code>	Orquestacion local de PostgreSQL y backend.
<code>docker-compose.override.yml</code>	Override automatico de desarrollo; activa <code>pnpm start:debug</code> para el backend.

3. Backend: `server/`

Tecnologias

- **Node.js 20+** y **TypeScript 5.5**.
- **NestJS 10** con Express como adaptador HTTP.
- **PostgreSQL 16** como base de datos.
- **Prisma 5.22** como ORM, cliente tipado y sistema de migraciones.
- **JWT, Passport y bcryptjs** para autenticacion y hashing.
- **Socket.IO 4** y `@nestjs/websockets` para tiempo real.
- **Swagger/OpenAPI** mediante `@nestjs/swagger`.
- **class-validator** y **class-transformer** para DTOs y validacion.
- **Helmet** y `@nestjs/throttler` para protecciones HTTP.
- **Nodemailer** y **Handlebars** para correo y plantillas.

- **Jest, Supertest, ts-jest, ESLint y Prettier** para pruebas y calidad.
- Gestor de paquetes: **pnpm 10**.

Estructura interna

Directorio	Responsabilidad
<code>server/src/auth/</code>	Registro, login, refresh tokens, recuperacion de contraseña, guards y estrategias Passport.
<code>server/src/vps/</code>	Alta, consulta y administracion de VPS; generacion y entrega del instalador/token del agente.
<code>server/src/agent/</code>	Endpoints consumidos por agentes, autenticacion de agentes y gateway del canal <code>/agents</code> .
<code>server/src/alertas/</code>	Creacion, consulta y actualizacion del estado de alertas.
<code>server/src/bloqueos/</code>	Bloqueo y desbloqueo de IPs a traves del agente.
<code>server/src/configuracion/</code>	Nivel de autonomia, umbrales y preferencias de notificacion por VPS.
<code>server/src/whitelist/</code>	Gestion de IPs permitidas.
<code>server/src/dashboard-gateway/</code>	Gateway Socket.IO para notificar eventos al frontend en <code>/dashboard</code> .
<code>server/src/cron/</code>	Tareas programadas del backend.
<code>server/src/health/</code>	Endpoint y comprobaciones de salud del servicio.
<code>server/src/mail/</code>	Envio de correo SMTP y plantillas Handlebars.
<code>server/src/prisma/</code>	<code>PrismaService</code> y encapsulacion del cliente de base de datos.
<code>server/src/common/</code>	Funcionalidad transversal: guards, filtros, decoradores y servicios compartidos.
<code>server/prisma/schema.prisma</code>	Modelo de datos y contrato ORM.
<code>server/prisma/migrations/</code>	Historial versionado de cambios de esquema.

<code>server/config/</code>	Configuracion y validacion de variables de entorno.
<code>server/test/</code>	Pruebas end-to-end y configuracion de Jest E2E.
<code>server/public/</code>	Recursos publicos servidos por el backend, incluido el instalador y releases del agente.

4. Frontend: `client/`

Tecnologias

- **Angular 21** con componentes standalone y signals.
- **TypeScript 5.9**.
- **SCSS** para estilos.
- **RxJS 7.8** para flujos asincronos.
- **Socket.IO Client 4.8** para alertas en tiempo real.
- **@hey-api/openapi-ts** y `@hey-api/client-fetch` para generar el cliente REST desde OpenAPI.
- **@fontsource** con Montserrat, Roboto y JetBrains Mono.
- **Vitest**, JSDOM y Angular CLI para pruebas.
- Gestor de paquetes: **npm 10**.

Estructura interna

Directorio	Responsabilidad
<code>client/src/app/core/</code>	Infraestructura compartida de la aplicacion, cliente API, guards y servicios singleton.
<code>client/src/app/core/api-client/</code>	Cliente REST autogenerado desde <code>docs/openapi.json</code> ; no debe editarse manualmente.
<code>client/src/app/core/services/</code>	Fachada de servicios para consumir la API y encapsular logica comun.
<code>client/src/app/core/guards/</code>	Proteccion de rutas y acceso autenticado.
<code>client/src/app/features/</code>	Funcionalidades organizadas por dominio: auth, dashboard, vps, alertas, bloqueos, configuracion, nosotros, shell y whitelist.

<code>client/src/app/shared/</code>	Componentes y utilidades reutilizables entre features.
<code>client/src/environments/</code>	Configuración por entorno, incluida la URL de la API.
<code>client/public/</code>	Imágenes e iconos estáticos.
<code>client/src/styles.scss</code>	Estilos globales y tema de la aplicación.
<code>client/angular.json</code>	Configuración de build, serve, pruebas, assets y reemplazos de entorno.

El cliente REST se regenera con `npm run api:generate` a partir del contrato OpenAPI. El canal WebSocket se configura aparte porque no forma parte del cliente REST generado.

5. Agente: `agent/`

Tecnologías

- **Python 3.11 o superior.**
- **psutil** para métricas y estado del sistema.
- **Requests** para comunicación HTTP con el backend.
- **python-socketio** para comunicación Socket.IO.
- **pytest** como dependencia opcional de desarrollo.
- Ejecución prevista como servicio **systemd** en cada VPS monitoreado.

Estructura interna

Directorio	Responsabilidad
<code>agent/src/lumi_agent/core/</code>	Configuración, logging, almacenamiento local, configuración del agente y clase base de monitores.
<code>agent/src/lumi_agent/monitors/</code>	Monitores de SSH, HTTPS y sistema. Detectan eventos y recopilan métricas.
<code>agent/src/lumi_agent/senders/</code>	Envío de alertas, métricas y heartbeat al backend.

<code>agent/src/lumi_agent/control/</code>	Cliente WebSocket y administracion local de UFW para aplicar acciones de red.
<code>agent/config/</code>	Configuracion TOML del agente y archivo de ejemplo.
<code>agent/main.py</code>	Punto de entrada del proceso del agente.
<code>agent/pyproject.toml</code>	Metadatos, version minima de Python y dependencias del proyecto.
<code>agent/requirements.txt</code>	Dependencias Python para instalaciones que usan pip.
<code>agent/Dockerfile</code>	Imagen del agente cuando se requiere empaquetarlo en contenedor.

El agente se instala fuera del Compose productivo del sistema central. El instalador y sus releases se publican desde el backend bajo `server/public/agent/`.

6. Despliegue e infraestructura: `deploy/`

Tecnologias y herramientas

- **Docker y Docker Compose** para empaquetar y orquestar servicios.
- **PostgreSQL 16 Alpine**.
- **Nginx** como reverse proxy y servidor frontal.
- Imagen de produccion de **NestJS** y build estatico de **Angular**.
- **Prisma Migrate** ejecutado por el servicio temporal `migrate` antes de iniciar el backend.
- **SMTP** para correo saliente configurado mediante variables de entorno.
- **TLS/HTTPS** previsto para Nginx; la configuracion base incluida es HTTP y debe completarse antes de exponer el sistema.

Archivos y responsabilidades

Ruta	Responsabilidad
<code>deploy/docker-compose.prod.yml</code>	Levanta postgres, migrate, backend, frontend y nginx en la red interna <code>lumi-net</code> . Solo Nginx publica el puerto de entrada.

deploy/nginx/Dockerfile	Construye la imagen del reverse proxy.
deploy/nginx/nginx.conf	Enruta / al frontend, /api/ y /agent/ al backend, y /socket.io/ con soporte de upgrade WebSocket.
deploy/.env.production.example	Plantilla de variables de produccion.
deploy/README.md	Procedimientos de despliegue local, staging, EC2/AWS, persistencia y checklist de seguridad.
client/Dockerfile	Construye y sirve el frontend Angular para produccion.
server/Dockerfile	Construye la imagen productiva del backend sin dependencias de desarrollo.

Desarrollo local

En la raiz, `docker-compose.yml` levanta PostgreSQL y el backend.

`docker-compose.override.yml` cambia el comando del backend a `pnpm start:debug`.

El frontend y el agente se ejecutan de forma independiente durante el desarrollo.

7. Herramientas adicionales y contratos

Ruta/herramienta	Uso
docs/openapi.json	Copia versionada del contrato REST; sirve como entrada para generar el cliente Angular.
test-ws/	Pruebas manuales de WebSocket con <code>dashboard-listener.js</code> y <code>mock-agent.js</code> ; usa su propio <code>package.json</code> .
docs/PRUEBAS_E2E.md	Casos y recorrido de pruebas end-to-end.
server/test/	Suite E2E del backend con Jest y Supertest.
server/prisma/	Esquema y migraciones de base de datos.
server/public/install.sh	Instalador del agente distribuido al VPS.

README.md de la raiz	Vision de producto, componentes, seguridad y arranque general.
docs/	Documentacion de integracion backend-agente, base de datos, configuracion, frontend y agente.

8. Resumen de tecnologias por proyecto

Proyecto	Lenguaje	Framework/runtime	Persistencia/comunicacion	Calidad y tooling
Backend	TypeScript	Node.js + NestJS + Express	PostgreSQL, Prisma, REST, Socket.IO, SMTP	Jest, Supertest, ESLint, Prettier, Swagger
Frontend	TypeScript	Angular	Fetch/OpenAPI, Socket.IO Client, RxJS	Angular CLI, Vitest, JSDOM, SCSS
Agente	Python	Python 3.11+	Requests, python-socketio, almacenamiento/configuracion local	pytest, pyproject.toml, requirements.txt
Despliegue	YAML, Nginx	Docker Compose + Nginx	Red Docker, volumen PostgreSQL, reverse proxy	Healthchecks, migraciones Prisma, variables .env
Pruebas WebSockets	JavaScript	Node.js	Socket.IO client/server de prueba	Scripts manuales en test-ws/